

Cross platform adventures and useful MSVC flags

John Franklin Rickard | john@moduleworks.com

Overview

- Preprocessor basics
- Compiler detection
- Symbol exports
- C++ standard helpers
- Compiler weirdness

Fair warning

Fair warning



Macrooos



Preprocessor basics

- PP = Preprocessor
- glorified text replacement with horrible rules
- runs before actual C++ compilation takes place
- mostly # + a PP directive
- can have parameters, values, certain expressions, ...

```
#define HELLO1  
#    define HELLO2  
#define HELLO3 15  
#define HELLO4 // This is a comment  
#define HELLO4(MSG) MSG // This is a comment
```

Preprocessor basics

- `#if, #else, #elif, #endif`
- `#define, #undef, #, ##`
- `#ifdef, ifndef, #ifndef, #elifndef(C++23!)`
- `#pragma, _Pragma, __pragma`
- `#error, #warning (C++23!)`
- `#line` (always forget about this one)

But why?!

- Cross platform and cross compiler development
- Different language standards
- Non Standard C++ or compiler extensions
- OS or Hardware APIs
- C++ just has a long history (in combination with C)

Slightly related [Youtube Docu Tip](#):

The Story of C++: The World's Most
Consequential Programming Language

Platform detection

```
#include <print>

void greet_user() {
    #if defined(WIN32)
        std::print("Hello there Windows user!");
    #elif __APPLE__
        std::print("Hello there Apple user!");
    #elif __linux__
        std::print("Hello there linux user!");
    //...
    #else
        #error "Since when do we support this platform?!"
    #endif
}
```

Done?

Platform detection

predef on github

.

Windows

Type	Macro	Description
Identification	<code>_WIN16</code>	Defined for 16-bit environments 1
Identification	<code>_WIN32</code>	Defined for both 32-bit and 64-bit environments 1
Identification	<code>_WIN64</code>	Defined for 64-bit environments 1
Identification	<code>__WIN32__</code>	Defined by Borland C++
Identification	<code>__TOS_WIN__</code>	Defined by xLC
Identification	<code>__WINDOWS__</code>	Defined for 16-bit-environments by Watcom C/C++ 1

Compiler detection

```
#if __clang__
    #if _MSC_VER
        "clang-cl"
    #elif __apple_build_version
        "apple clang"
    #else
        "clang"
    #endif
#elif _MSC_VER
    "MSVC"
#elif __GNUC__
    "GCC"
#endif
```

Compiler detection

Recommendations:

- check if/else chains exhaustively
- it never stays at one OS or compiler
- verify check order works for your use case
- #error in all other cases, future you will thank you!
- check what combinations you actually need

Compiler version ranges

```
#if _MSC_VER > 1900
    // use shiny new MSVC feature
#else
    // use fallback
#endif
```

```
#if _MSC_VER < 2000
    // something MSVC specific
#else
    // use normal code -- woops
#endif
```

Compiler version ranges

```
#pragma once // this is "standard", fight me :)

#if defined(_MSC_VER) && _MSC_VER < 1700
    // use bug workaround for msvc-like compilers
#else
    // use default
#endif
```

Greetings!

```
// greet.hpp  
void greet();
```

```
// greet.cpp  
void greet() { std::print("Hello C++ User Group!"); }
```

```
// main.cpp  
int main() { greet(); }
```

Greetings!

```
// greet.hpp  
#pragma once  
void greet();
```

```
// greet.cpp  
#include "greet.hpp"  
#include <print>  
void greet() { std::print("Hello C++ User Group!"); }
```

```
// main.cpp  
#include "greet.hpp"  
int main() { greet(); }
```

Greetings!

```
// greet.hpp
#pragma once
void greet();

// greet.cpp
#include "greet.hpp"
#include <print>
void greet() { std::print("Hello C++ User Group!"); }

// main.cpp
#include "greet.hpp"
int main() { greet(); }
```

```
<source>:2:(.text+0x5): undefined reference to `greet()'
```

Greetings SHARED!

```
cmake_minimum_required(VERSION 4.4)
project(complicated_greeting LANGUAGES C++)
set(CMAKE_CXX_VISIBILITY_PRESET hidden)
add_library(lib_greet SHARED) # <---
target_sources(lib_greet
    PUBLIC
        FILE_SET HEADERS FILES greet.hpp
    PRIVATE
        FILE_SET SOURCES FILES greet.cpp
)
add_executable(run_greeter main.cpp)
target_link_libraries(run_greeter
    PRIVATE
        lib_greet
)
```

Symbol exports

```
// greet.hpp
#pragma once
__attribute__((visibility("default"))) void greet();

// greet.cpp
#include "greet.hpp"
#include <print>
void greet() { std::print("Hello C++ User Group!"); }

// main.cpp
#include "greet.hpp"
int main() { greet(); }
```

Hello C++ User Group!

Symbol exports

```
#if defined(_MSC_VER)
    #define SYMBOL_EXPORT __declspec(dllexport)
    #define SYMBOL_IMPORT __declspec(dllimport)
#elif defined(__GNUC__) || defined(__clang__)
    #define SYMBOL_EXPORT __attribute__((visibility("default")))
    #define SYMBOL_IMPORT __attribute__((visibility("default")))
#else
    #error Unsupported compiler
#endif
```

Symbol exports

You can use `__declspec` in gcc if you want 😊

```
'__declspec' attributes are not enabled;  
use '-fdeclspec' or '-fms-extensions'  
to enable support for __declspec attributes
```

Symbol exports

```
#ifdef LIB_GREETING_STATIC
    #define MY_API
#elifdef LIB_GREETING_EXPORT
    #define MY_API SYMBOL_EXPORT
#else
    #define MY_API SYMBOL_IMPORT
#endif
```

Symbol exports

```
// greet.hpp  
#pragma once  
#include "export.hpp"  
MY_API void greet();
```

```
// greet.cpp  
#include "greet.hpp"  
#include <print>  
void greet() { std::print("Crossplatform greetings!"); }
```

```
// main.cpp  
#include "greet.hpp"  
int main() { greet(); }
```

C++ standard helpers

- `__cplusplus`
- `__has_include` (C++17)
- `__has_cpp_attribute` (C++20)
- `__has_embed` (C++26!)
- language feature-test macros (~C++14 onwards)
- `<version>` standard header containing library feature-test macros (C++20)

C++ attribute detection

```
#ifdef __has_cpp_attribute
    #if __has_cpp_attribute(noreturn)
        #define VALUE 1
    #else
        #define VALUE 0
    #endif
#else
    #define VALUE 0
#endif

int main() { return VALUE ; }
```

Side note

```
// useful until C++20 is your minimum
// or all supported compilers have it built in
#ifdef __has_cpp_attribute
    #define HAS_CPP_ATTRIBUTE(attr) __has_cpp_attribute(attr)
#else
    #define HAS_CPP_ATTRIBUTE(attr) 0
#endif
```

C++ attribute detection

```
#if __has_cpp_attribute(noreturn)
    #define VALUE 1
#else
    #define VALUE 0
#endif

int main() { return VALUE ; }
```

Compiler	Version	Returns
gcc	16.1	?
clang	22.1.0	?
msvc	v19.50	?

C++ attribute detection

```
#if __has_cpp_attribute(noreturn)
    #define VALUE 1
#else
    #define VALUE 0
#endif

int main() { return VALUE ; }
```

Compiler	Version	Returns
gcc	16.1	1
clang	22.1.0	1
msvc	v19.50	1

C++ attribute detection

```
#if __has_cpp_attribute(likely)
    #define VALUE 1
#else
    #define VALUE 0
#endif

int main() { return VALUE ; }
```

Compiler	Version	Returns
gcc	16.1	?
clang	22.1.0	?
msvc	v19.50	?

C++ attribute detection

```
#if __has_cpp_attribute(likely)
    #define VALUE 1
#else
    #define VALUE 0
#endif

int main() { return VALUE ; }
```

Compiler	Version	Returns
gcc	16.1	1
clang	22.1.0	1
msvc	v19.50	0

C++ attribute detection

```
#if __has_cpp_attribute(likely)
    #define VALUE 1
#else
    #define VALUE 0
#endif

int main() { return VALUE ; }
```

Compiler	Version	Returns
gcc	16.1	1
clang	22.1.0	1
msvc	v19.50 with /std:c++20	1

C++ attribute detection

```
#if __has_cpp_attribute(no_unique_address)
    #define VALUE 1
#else
    #define VALUE 0
#endif

int main() { return VALUE ; }
```

Compiler	Version	Returns
gcc	16.1	?
clang	22.1.0	?
msvc	v19.50	?

C++ attribute detection

```
#if __has_cpp_attribute(no_unique_address)
    #define VALUE 1
#else
    #define VALUE 0
#endif

int main() { return VALUE ; }
```

Compiler	Version	Returns
gcc	16.1	?
clang	22.1.0	?
msvc	v19.50 with /std:c++20	?

C++ attribute detection

```
#if __has_cpp_attribute(no_unique_address)
    #define VALUE 1
#else
    #define VALUE 0
#endif

int main() { return VALUE ; }
```

Compiler	Version	Returns
gcc	16.1	1
clang	22.1.0	1
msvc	v19.50 with /std:c++20	0

C++ attribute detection

“Small” addition:

```
[[no_unique_address]] is ignored by MSVC even in C++20 mode;  
instead, [[msvc::no_unique_address]] is provided.
```

cppreference

C++ attribute detection

```
#if __has_cpp_attribute(no_unique_address)
    #ifdef _MSC_VER
        #define NO_UNIQUE_ADDRESS [[mscv::no_unique_address]]
    #else
        #define NO_UNIQUE_ADDRESS [[no_unique_address]]
    #endif
#else
    #define NO_UNIQUE_ADDRESS
#endif
```

C++ header detection

```
#include <string>
void Read(const std::string& input);

#ifdef __has_include
    #if __has_include(<string_view>)
        #include <string_view>
        void Read(std::string_view input);
    #endif
#endif
```

C++ header detection

```
#include <vector>
int Sum(const std::vector<int>& input);

#ifdef __has_include
    #if __has_include(<span>)
        #include <span>
        int Sum(std::span<int> input);
    #endif
#endif
```

C++ header detection

```
#include <vector>
int Sum(const std::vector<int>& input);

#ifdef __has_include
    #if __has_include(<span>)
        #include <span>
        int Sum(std::span<int> input);
    #endif
#endif
```

```
warning STL4038: The contents of <span> are available
    only with C++20 or later.
<source>(6): error C2039: 'span': is not a member of 'std'
```

C++ header detection

```
int Sum(const std::vector<int>& input);

// predefined long of WG21 meeting with
// YYYYMM format when the standard draft
// got completed and sent out for approval.
// C++20 -> completed in 2020-02 Prague
#if __cplusplus >= 202002L
    #include <span>
    int Sum(std::span<int> input);
#endif
```

Feature-test macros

- allows checks of very specific features
- if existent, extends to YYYYMM date when it was voted into working draft (as long)
- easy to reason about and lookup
- requirement for all future C++ standard additions

cppreference

C++ language features

```
// check relaxed constexpr rules
#if __cpp_constexpr >= 201304L
    #define CONSTEXPR constexpr
#else
    #define CONSTEXPR inline
#endif

CONSTEXPR int foo() {
    const int i = 42;
    return i;
}
```

C++ library features

```
// useful in a common PP header!
#ifdef __has_include
    #if __has_include(<version>)
        #include <version>
    #endif
#endif

auto concat(std::string s1, std::string_view s2) {
    // C++26 finally ships operator + for string view!
    #if __cpp_lib_string_view >= 202403L
        return s1 + s2;
    #else // fallback
        return s1 + std::string(s2);
    #endif
}
```

Compiler weirdness



Yaaaaaaay



C++ compiler extensions

-  Set Standard explicitly and disable extensions
- Most compilers default to extensions enabled
- GCC 16 defaults to `-std=gnu++20`
(C++20 with gnu extensions)
- MSVC defaults to non standard C++
disable this with `/permissive-`

Clang weirdness

clang/clang-cl emits compiler defines of other common compilers.

Makes it complicated to check preprocessor flags.

Apple clang weirdness

We had regular problems with fwd declarations there that did not have all the decorators.

```
struct Important;  
// ...  
struct MW_API [[nodiscard]] Important {};
```

GCC weirdness

```
#define MW_API __attribute__((visibility("default")))  
  
MW_API [[deprecated]] void LegacySetup();
```

GCC weirdness

```
#define MW_API __attribute__((visibility("default")))  
  
struct MW_API [[deprecated]] LegacyFoo{};
```

gcc (here 12.5) fails with parser(!) error

```
<source>:3:15: error: expected identifier before '[' token  
  3 | struct MW_API [[deprecated]] LegacyFoo{};  
    |               ^
```

GCC weirdness

Found no alternative syntax mentioned in the gcc documentation, although I knew there was one.

Most google answers point to

`__attribute__((visibility("default")))`
answers.

LLM gave the answer directly, but no idea where to find all the definitions.

GCC weirdness

```
#define MW_API [[gnu::visibility("default")]]  
  
struct MW_API [[deprecated]] LegacyFoo{};
```

Works again 🎉 🎉 🎉

Mixing of old and new attribute style is not always supported, choose one!

MSVC weirdness

```
#include <iostream>
int main() { std::cout << __cplusplus; }
```

Compiler	Version	Returns
gcc	16.1	?
clang	22.1.0	?
clang-cl	22.1.0	?
msvc	v19.50	?

MSVC weirdness

```
#include <iostream>
int main() { std::cout << __cplusplus; }
```

Compiler	Version	Returns
gcc	16.1	202002
clang	22.1.0	201703
clang-cl	22.1.0	201402
msvc	v19.50	199711

MSVC weirdness

Learn > C++, C, and Assembler >

 Ask Learn

`/Zc:__cplusplus` (Enable updated `__cplusplus` macro)



Summarize this article for me

The `/Zc:__cplusplus` compiler option enables the `__cplusplus` preprocessor macro to report an updated value for recent C++ language standards support. By default, Visual Studio always returns the value `199711L` for the `__cplusplus` preprocessor macro.

MSVC weirdness

For finer-grained detection of changes to the compiler toolset, use the `_MSC_VER` predefined macro. The value of this built-in macro is incremented for every toolset update in Visual Studio 2017 and later versions. The `_MSVC_LANG` predefined macro reports the standard version whether the `/Zc:__cplusplus` option is enabled or disabled. When `/Zc:__cplusplus` is enabled, `__cplusplus` has the same value as `_MSVC_LANG`.

MSVC weirdness

Option	Behavior
<code>/Zc:__cplusplus[-]</code>	Enable the <code>__cplusplus</code> macro to report the supported standard. Off by default.
<code>/Zc:__STDC__</code>	Enable the <code>__STDC__</code> macro to report the C standard is supported. Off by default.
<code>/Zc:alignedNew[-]</code>	Enable C++17 over-aligned dynamic allocation. Off by default unless <code>/std:c++17</code> or later is specified.
<code>/Zc:auto[-]</code>	Enforce the new Standard C++ meaning for <code>auto</code> . On by default.
<code>/Zc:char8_t[-]</code>	Enable or disable C++20 native <code>u8</code> literal support as <code>const char8_t</code> . Off by default unless <code>/std:c++20</code> or later is specified.
<code>/Zc:checkGwOdr[-]</code>	Enforce Standard C++ One-Definition-Rule (ODR) violations under <code>/Gw</code> .
<code>/Zc:enumTypes[-]</code>	Enable Standard C++ rules for <code>enum</code> type deduction. Off by default.
<code>/Zc:externC[-]</code>	Enforce Standard C++ rules for <code>extern "C"</code> functions. Off by default unless <code>/permissive-</code> is specified.
<code>/Zc:externConstexpr[-]</code>	Enable external linkage for <code>constexpr</code> variables. Off by default unless <code>/permissive-</code> is specified.
<code>/Zc:forScope[-]</code>	Enforce Standard C++ <code>for</code> scoping rules. On by default.
<code>/Zc:gotoScope[-]</code>	Enforce Standard C++ <code>goto</code> rules around local variable initialization. Off by default unless <code>/permissive-</code> is specified.
<code>/Zc:hiddenFriend[-]</code>	Enforce Standard C++ hidden friend rules. Off by default unless <code>/permissive-</code> is specified.

MSVC weirdness

`/Zc:throwingNew` (Assume operator new throws)



Summarize this article for me

When the `/Zc:throwingNew` option is specified, the compiler optimizes calls to `operator new` to skip checks for a null pointer return. This option tells the compiler to assume that all linked implementations of `operator new` and custom allocators conform to the C++ standard and throw on allocation failure. By default in Visual Studio, the compiler pessimistically generates null checks (`/Zc:throwingNew-`) for these calls, because users can link with a non-throwing implementation of `operator new` or write custom allocator routines that return null pointers.

MSVC weirdness

If `/Zc:inline` is specified, the compiler enforces the C++11 requirement that all functions declared `inline` must have a definition available in the same translation unit if they're used. When the option isn't specified, the Microsoft compiler allows non-conformant code that invokes functions declared `inline` even if no definition is visible. For more information, see the C++11 standard, in section 3.2 and section 7.1.2. This compiler option was introduced in Visual Studio 2013 Update 2.

**Thank you for your
attention!**